



QUESTIONS

2 / 2 completed

Sahred

SAHRED

Write a Java program to demonstrate how to use synchronization to prevent multiple threads from accessing a shared resource simultaneously.

PROGRAM EDITOR

Java

Run

Save

```
9      public void run() {  
10          for (int i = 0; i < 10000; i++) {  
11              increment();  
12          }  
13      }  
14  }  
15  
16  public static void main(String[] args) throws Exception {  
17  
18      Thread t1 = new MyThread();  
19      Thread t2 = new MyThread();  
20  
21      t1.start();  
22      t2.start();  
23  
24      t1.join();  
25      t2.join();  
26  
27      System.out.println("Shared value: " + sharedValue);  
28  }  
29 }
```

OUTPUT

Shared value: 20000

INPUT

Your INPUT goes here!



QUESTIONS

2 / 2 completed

AtomicInteger

ATOMICINTEGER

Write a Java program to demonstrate how to use the AtomicInteger class to provide atomic access to a shared integer variable.

Output:

Shared value: 20000

This confirms that the two threads were able to increment the shared value atomically and the final value of the shared value is as expected. The exact number may vary as it depends on the scheduling of the threads, but it should always be 20000.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.concurrent.atomic.AtomicInteger;
2
3 class Main {
4     public static void main(String[] args) throws Exception {
5
6         AtomicInteger shared = new AtomicInteger(0);
7
8         Thread t1 = new Thread(() -> {
9             for (int i = 0; i < 10000; i++) {
10                 shared.incrementAndGet();
11             }
12         });
13
14         Thread t2 = new Thread(() -> {
15             for (int i = 0; i < 10000; i++) {
16                 shared.incrementAndGet();
17             }
18         });
19
20         t1.start();
21         t2.start();
22     }
23 }
```

OUTPUT

Shared value: 20000

INPUT

Your INPUT goes here!